HOTEL MANAGEMENT SYSTEM

Deployment on Microsoft Azure

Project Report

22AIE303 · Cloud Computing

Team Number: 03

Roll Number	Name
CB.SC.U4AIE24116	J. Lakshmi Tejaswi
CB.SC.U4AIE24130	Vishnu Teja
CB.SC.U4AIE24135	Mithul Akshay
CB.SC.U4AIE24136	Mohan Innani

Amrita Vishwa Vidyapeetham, Coimbatore

Contents

1. Introduction 3
2. Problem Statement 3
3. Objectives 4
4. Cloud Service and Deployment Models 4
5. Proposed System Architecture 6
6. Azure Services Used 8
7. Infrastructure as Code 9
8. Database Migration Strategy 11
9. Scalability and Elasticity 12
10. High Availability and Fault Tolerance 13
11. Security Design 14
12. Monitoring and Cost Model 15
13. Implementation Status and Plan 16
14. Challenges and Mitigations 17
15. Conclusion and Future Scope 18
16. Repository and Appendix 19

1. Introduction

The Hotel Management System is a three-tier database application developed for the Database Management Systems course. It manages guest registration, room inventory, bookings, check-in and check-out, billing and payments for a small hotel, replacing the paper registers such a hotel would otherwise keep.

The system consists of a Streamlit web interface written in Python, a data access layer in which every SQL statement in the project is defined, and a MySQL database of six normalised tables with foreign keys, views, triggers and a stored procedure. It is complete and working.

In its present form the application runs on a single machine. That arrangement is adequate for a demonstration but unsuitable for a working hotel: the machine is a single point of failure, the system is reachable only from that one desk, and its capacity is fixed by the hardware it happens to run on.

This project migrates the system to Microsoft Azure. The objective is not to rewrite the application but to place it on infrastructure that is highly available, reachable from anywhere, elastic with respect to demand, and defined entirely as code so that it can be created and destroyed reproducibly.

2. Problem Statement

Six specific limitations motivate the migration.

Limitation	Consequence
Single point of failure	Application and database share one machine. If it fails the hotel cannot take a booking until it is repaired.
No remote access	Staff can only work at that one computer; a manager cannot check occupancy from elsewhere.
Fixed capacity	Hardware is sized for peak season and idles the rest of the year. A surge in demand requires buying a larger machine.
Manual backups	Backups depend on someone remembering to take them. A disk failure loses every guest and payment record.
Capital expenditure	Servers and power protection must be purchased before the first guest is served, and replaced every few years.
Maintenance burden	Patching the operating system, the database engine and the network is the hotel's responsibility rather than a specialist's.

A further problem is specific to the existing implementation: the database password is stored in plain text in a configuration file inside the source repository. Any migration that does not address this simply relocates the weakness.

3. Objectives

1. Migrate the application and its MySQL database to a public cloud platform.
2. Replace the self-hosted database with a managed service, so that patching, backup and failover become the provider's responsibility.
3. Introduce elastic scaling, so the number of application servers follows demand rather than a purchasing decision.
4. Deploy across two Availability Zones, so the loss of one datacentre does not take the system down.
5. Remove every credential from the source repository, holding secrets in a managed vault and retrieving them with an identity rather than a key.
6. Define the entire environment as infrastructure code, so that it can be reviewed, versioned, destroyed and rebuilt identically.
7. Monitor the deployment and analyse its running cost against the alternative of owning hardware.

4. Cloud Service and Deployment Models

4.1 Service models

Cloud services are conventionally divided into three layers, distinguished by how much of the stack the provider manages. This project deliberately uses all three.

Model	What the provider manages	Where it appears in this project
IaaS	Virtual machines, storage, networking. The customer manages the operating system and everything above it.	Azure Virtual Machine Scale Sets run the application. We choose Ubuntu 22.04, install Python and Streamlit, and maintain them.
PaaS	Everything IaaS covers, plus the runtime or database engine itself.	Azure Database for MySQL Flexible Server. Azure patches the engine, takes backups and performs failover; we design only the schema and the queries.
SaaS	The complete application. The customer opens a browser and uses it.	The finished system is delivered this way to hotel staff, who visit a URL and own no infrastructure at all.

The general trade-off is that the higher a component sits in this stack, the less the customer manages and the less the customer controls. Choosing PaaS for the database means accepting Azure's patching schedule in exchange for never having to patch MySQL again.

4.2 Deployment models

Deployment models describe who the infrastructure is shared with.

Model	Description	Assessment for this project
Public cloud	Infrastructure owned by a provider and shared among many tenants over the internet.	Chosen. No capital expenditure, Availability Zones available immediately, and resources can be created and destroyed as the project requires.
Private cloud	Infrastructure dedicated to a single organisation, on its own premises or hosted for it.	Rejected. The capital cost the migration is meant to avoid returns, and a single hotel has no regulation requiring it.
Hybrid cloud	Sensitive data kept private while general workloads run in the public cloud.	Rejected as unnecessary complexity at this scale, though it would be the right answer if guest

Model	Description	Assessment for this project
		identity documents were legally required to stay on site.
Community cloud	Shared between organisations with common requirements.	Not applicable to a single independent hotel; it would suit a hotel group sharing a booking platform.

5. Proposed System Architecture

The deployment places the three application tiers inside a single virtual network, with each tier in its own subnet and its own set of firewall rules. Only the entry point is reachable from the internet.

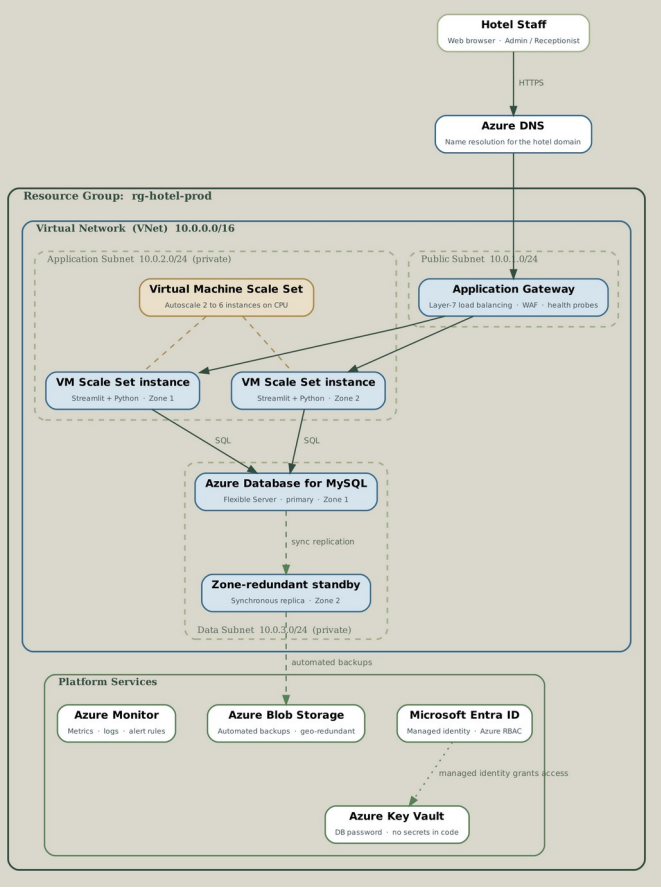


Figure 1 - Azure deployment architecture. Resource group, virtual network, three subnets and supporting platform services.

5.1 Request path

A member of hotel staff opens the application in a browser. Azure DNS resolves the hotel's domain to the public address of an Application Gateway, which sits in the only publicly addressable subnet. The gateway terminates the connection, checks which application instances are responding to its health probe, and forwards the request to one of them.

The application instances live in a private subnet and have no public IP address of any kind. They query the database over the virtual network. The database likewise has no public endpoint: it is integrated into a delegated subnet and resolved through a private DNS zone, so the hostname only resolves to a private address and only from inside the network.

5.2 Tier summary

Tier	Subnet	Components
Entry	10.0.1.0/24 (public)	Application Gateway v2 with health probes, zone-redundant public IP
Application	10.0.2.0/24 (private)	Virtual Machine Scale Set, 2-6 Ubuntu instances across Zones 1 and 2

Tier	Subnet	Components
Data	10.0.3.0/24 (private, delegated)	Azure Database for MySQL Flexible Server, with optional zone-redundant standby
Platform	outside the VNet	Key Vault, Azure Monitor, Blob Storage, Microsoft Entra ID

6. Azure Services Used

Twelve services make up the deployment. Each is listed with the function it performs and the cloud computing concept it demonstrates.

Service	Function	Concept
Virtual Machine Scale Sets	Run the Streamlit application across two zones	IaaS, elasticity
Azure Database for MySQL	Managed MySQL 8.0, Flexible Server tier	PaaS, managed services
Application Gateway v2	Layer-7 load balancing, health probing, session affinity	Load balancing
Azure Virtual Network	Private network with three isolated subnets	Network isolation
Network Security Groups	Per-tier stateful firewall rules	Defence in depth
Azure Key Vault	Stores the database password	Secrets management
Microsoft Entra ID	Managed identity and Azure RBAC role assignment	Identity and access management
Azure Monitor	Metrics, logs, alert rules and action groups	Observability
Azure Blob Storage	Holds automated database backups	Durable object storage
Availability Zones	Two physically separate datacentres in one region	Fault tolerance
Azure Bicep	Declares every resource above as code	Infrastructure as code
Azure for Students	Free credit with spending limit	Pay-as-you-go pricing

7. Infrastructure as Code

The entire environment is declared in Azure Bicep rather than created by hand in the Azure portal. This is the central engineering decision of the project and deserves justification.

A portal-built environment exists only as a sequence of remembered actions. It cannot be reviewed, cannot be version-controlled, and cannot be recreated reliably after it is deleted. Because an idle Application Gateway and database continue to bill by the hour, a student project genuinely needs to destroy its environment between demonstrations, which makes reproducibility a practical requirement rather than a stylistic preference.

7.1 File structure

File	Responsibility
`main.bicep`	Orchestrates all six modules, wires their outputs together, and renders the cloud-init document
`modules/network.bicep`	Virtual network, three subnets, three network security groups, subnet delegation
`modules/keyvault.bicep`	Key Vault with RBAC authorisation, network ACLs, and the password secret
`modules/database.bicep`	MySQL Flexible Server, private DNS zone and link, server parameters, the application database
`modules/gateway.bicep`	Application Gateway v2, zone-redundant public IP, listener, probe, routing rule
`modules/compute.bicep`	Scale set, system-assigned identity, Key Vault role assignment, autoscale rules
`modules/monitoring.bicep`	Log Analytics workspace, action group, metric alert rule
`cloud-init.yaml`	Bootstraps each VM: packages, secret retrieval, application install, systemd service
`deploy.sh`	Creates the resource group, validates, deploys, then loads the schema
`load-schema.sh`	Imports the SQL schema from inside the virtual network
`destroy.sh`	Removes the resource group and everything in it, with confirmation

7.2 Example: the autoscale rule

The elasticity claimed in section 9 is not a description of intent; it is this declaration.

```
resource autoscale 'Microsoft.Insights/autoscalesettings@2022-10-01' = {
  properties: {
    profiles: [ {
      capacity: { minimum: '2', maximum: '6', default: '2' }
      rules: [ {
        metricTrigger: {
          metricName: 'Percentage CPU'
          timeWindow:  'PT5M'
          operator:    'GreaterThan'
          threshold:   70
        }
```

```
        scaleAction: {
          direction: 'Increase'
          value:     '1'
          cooldown:  'PT5M'
        } } ] } ] } }
```

7.3 Validation

Infrastructure code that has never been compiled is no more trustworthy than a diagram. The templates were therefore checked before submission:

- All seven Bicep files compile with the Bicep CLI to a single valid ARM template of approximately 59 KB, with no errors and no warnings. This verifies resource types, API versions, required properties, parameter types and the wiring between modules.
- The three shell scripts pass shellcheck at warning severity with no findings.
- The cloud-init document parses as valid YAML, and a check confirms that every placeholder it contains is substituted by main.bicep, so none can reach a virtual machine unreplaced.

Compilation does not prove that a deployment will succeed against a live subscription - quota limits, regional capacity and policy restrictions can still intervene - but it eliminates the entire class of errors that would otherwise only be discovered minutes into a deployment.

8. Database Migration Strategy

The database is migrated by a lift-and-shift procedure. Because Azure Database for MySQL runs genuine MySQL 8.0, the schema transfers without modification; no query in the application changes.

1. Export the existing schema and data with mysqldump. The output is a single file containing all six tables, three views, two triggers and one stored procedure.
2. Provision the Flexible Server inside the delegated data subnet, with a private DNS zone so that it has no public endpoint.
3. Set the server parameter log_bin_trust_function_creators to ON before importing.
4. Import the dump from inside the virtual network.
5. Repoint the application. The configuration file is written at boot from Key Vault with the Azure hostname, so no code change is required.
6. Verify by re-running the CRUD operations and the four reports, and comparing row counts against the source database.

8.1 Why step three is necessary

Azure Database for MySQL enables binary logging by default. With binary logging active, creating a stored function, procedure or trigger requires the SUPER privilege, which the administrator account on a managed server does not hold. Without the parameter change the import proceeds through the table definitions and then fails when it reaches the first trigger, leaving a half-populated database.

Setting log_bin_trust_function_creators to ON tells the server to trust routines created by the administrator. It is declared in database.bicep, so it is applied as part of provisioning rather than discovered during a demonstration.

8.2 Loading the schema into a private database

A consequence of giving the database no public endpoint is that the schema cannot be loaded from a laptop, or from Cloud Shell, or from anywhere else outside the virtual network. This is the intended security property, not an oversight, but it has to be worked around.

The project loads the schema by asking an application instance, which is already inside the network, to perform the import. The Azure CLI command az vmss run-command invoke executes a script through the virtual machine agent, requiring no SSH access, no public IP address and no bastion host. The instance already holds the repository and can read the database password from Key Vault with its managed identity, so no credential travels with the command.

9. Scalability and Elasticity

Scalability is the ability of a system to grow. Elasticity is the automation of that growth, in both directions, in response to live demand. The distinction matters: a system can be scalable and still require an engineer to act.

9.1 Vertical and horizontal scaling

Approach	Description	Use in this project
Vertical (scale up)	Give the existing machine more CPU or memory. Simple, but bounded by the largest available machine and usually requiring a restart.	Applied to the database. The server can be resized to a larger tier if query latency becomes the constraint.
Horizontal (scale out)	Add further machines and distribute load between them. No practical ceiling, and instances join while the system is running.	Applied to the application tier, where the scale set adds and removes instances automatically.

9.2 The autoscale policy as deployed

Condition	Action
Normal load	Two instances run, one in each Availability Zone
Average CPU above 70 per cent for 5 minutes	One instance is added, to a maximum of six
Average CPU below 30 per cent for 10 minutes	One instance is removed, never falling below two
Instance fails its health probe	The gateway stops routing to it and the scale set replaces it
Cooldown	Five minutes after scaling out, ten after scaling in

The asymmetry is deliberate. Scaling out uses a shorter window so that the system reacts quickly to load; scaling in uses a longer window and a longer cooldown so that a brief lull does not remove capacity that is about to be needed again. Without this the policy would oscillate.

10. High Availability and Fault Tolerance

Failure	System response
One application instance fails	The gateway's health probe stops routing to it within seconds; the scale set launches a replacement. Users are unaffected.
An entire Availability Zone fails	The instance and the database standby in the surviving zone continue serving. Capacity halves but the system stays available.
The primary database fails	Azure promotes the zone-redundant standby. The server hostname is unchanged, so the application reconnects without any code change.
Data deleted or corrupted in error	Restore from an automated backup. Point-in-time restore covers the retention window, seven days by default.

10.1 Recovery objectives

Two measures describe the quality of a recovery design. The recovery time objective (RTO) is how long the service may be unavailable. The recovery point objective (RPO) is how much data may be lost. Zone failover of a Flexible Server completes in minutes, and because replication to the standby is synchronous, the RPO for a zone failure is effectively zero.

These figures do not apply to accidental deletion. If a booking is deleted by mistake, the loss window is whatever has happened since the last restore point, and recovery means restoring to a new server rather than failing over.

10.2 An honest limitation

Zone-redundant high availability requires a General Purpose compute tier. The Burstable tier that an Azure for Students subscription can realistically afford does not support it. The deployed student configuration therefore runs without database high availability, while the architecture and the template provide for it.

Rather than leaving this ambiguous, database.bicep derives the tier from the chosen SKU, silently declines to request high availability when the tier cannot support it, and exposes an output stating whether it is genuinely active. The deployment script reads that output and prints a warning. The design is honest about which configuration is running.

11. Security Design

Security is applied in layers, from the network boundary inward.

11.1 Network isolation

Only the Application Gateway occupies a publicly addressable subnet. The application instances have no public IP address, and the database has no public endpoint at all. An attacker who discovers the database hostname cannot resolve it from outside the virtual network, because it is served by a private DNS zone linked only to that network.

11.2 Network security groups

Each subnet carries its own rules, so a compromise of one tier does not grant free movement to the next.

Subnet	Inbound rule
Gateway	TCP 80 and 443 from the internet; TCP 65200-65535 from GatewayManager, which Application Gateway v2 requires
Application	TCP 8501 from the gateway subnet only; all other inbound traffic denied
Data	TCP 3306 from the application subnet only

11.3 Secrets and identity

The weakness identified in section 2 is addressed directly. The database password is generated at deployment time, passed into the template as a secure parameter so that it does not appear in deployment logs, and stored in Key Vault. It is never written to a file on the deploying machine.

Each virtual machine carries a system-assigned managed identity. At boot it requests a token from the instance metadata endpoint, which is reachable only from inside the machine, and uses that token to read the secret. The identity is granted exactly one role, Key Vault Secrets User, scoped to the single vault. There is no key or connection string anywhere that could be committed to the repository or leaked.

The configuration file written at boot is owned by an unprivileged service account and readable only by it. The application itself never runs as root.

11.4 Shared responsibility

Azure is responsible for the security of the cloud: the physical datacentres, the hypervisor, and the managed services it operates. The customer remains responsible for security in the cloud: data, access rules, network configuration and application code. Choosing PaaS for the database moves the engine patching to Azure, but who may read the guests table remains entirely our decision.

Carried over from the database project, user passwords are stored as SHA-256 hashes and every query is parameterised, so the application is not vulnerable to SQL injection.

12. Monitoring and Cost Model

12.1 Monitoring

Signal	Purpose
CPU utilisation	Drives the autoscale rules, and raises an alert if it stays above 80 per cent for ten minutes
Health probe status	Shows which instances the gateway has removed from rotation
Database connections	Warns before the server's connection limit is reached
Storage consumed	Alerts before the database runs out of space
Application logs	Collected centrally, so a failure can be investigated without connecting to a machine

12.2 Cost model

Azure bills each component separately and by consumption.

Component	Basis of charge
Compute	Per second a virtual machine runs. Scaling in at night therefore costs nothing to do.
Database	Per hour by compute tier, plus the storage provisioned.
Application Gateway	Per hour plus capacity units consumed. This is the largest fixed cost in the design.
Storage	Per gigabyte per month. Backup snapshots are incremental.
Data transfer	Inbound free; outbound charged per gigabyte.

12.3 Capital versus operational expenditure

Buying a server is a capital expense: paid once, owned, and depreciating whether it is used or not. Cloud consumption is an operational expense: paid monthly, matched to actual usage, and stopped the moment a resource is switched off. For a hotel with a seasonal occupancy pattern, the second model follows demand far more closely than a purchasing cycle can.

It does not follow that the cloud is always cheaper. For a steady, always-on workload with predictable load, owning hardware can be less expensive over a three to five year horizon. The cloud wins where demand is variable, where capital is scarce, and where the alternative involves replacing hardware on a cycle. Claiming a universal cost advantage would be inaccurate.

For this project specifically, the practical cost control is the teardown script. An Application Gateway left running over a weekend consumes a meaningful share of a student subscription, so the environment is created for a demonstration and destroyed afterwards.

13. Implementation Status and Plan

13.1 Completed

- The three-tier application is built and working: Streamlit front end, Python data access layer, MySQL database.
- Six normalised tables in third normal form, with foreign keys, three views, two triggers and a stored procedure.
- The cloud architecture is designed: network layout, subnet plan, service selection, scaling policy and security model.
- The infrastructure is written as code: seven Bicep files, a cloud-init bootstrap document, and deployment, schema-loading and teardown scripts.
- The templates are validated. The full stack compiles to a valid ARM template with no errors or warnings; the scripts pass shellcheck; the cloud-init document parses and its placeholder substitution is verified complete.
- The migration procedure is documented, including the server parameter the import depends on.

13.2 Not yet done

The environment has not been provisioned against a live subscription. What exists is a complete infrastructure definition that creates it in a single command, rather than a manually assembled environment that cannot be recreated.

This was a deliberate allocation of effort. Building the environment through the portal would have produced screenshots sooner; building it as validated code produces something that can be deployed, destroyed and redeployed on demand, and that can be read and assessed as engineering work.

13.3 Remaining phases

Phase	Work
1	Provision the resource group and run the deployment against the subscription
2	Load the schema and verify row counts against the local database
3	Confirm the Application Gateway reports both instances healthy
4	Load-test the autoscale policy and capture the instance count rising and falling
5	Record the observed running cost, then tear the environment down

14. Challenges and Mitigations

Each of the following was identified while writing the templates, and each has a mitigation already present in the code.

Challenge	Mitigation
Streamlit holds session state in memory on the instance that served the first request. Behind a load balancer, a user sent to a different instance mid-session would be logged out.	Cookie-based session affinity is enabled on the Application Gateway backend settings, so a session stays on one instance.
Burstable database tiers cannot run zone-redundant high availability, but are the only tiers a student subscription can afford.	The template derives the tier from the SKU, declines high availability when it cannot be supported, and reports which mode is actually active.
Giving the database no public endpoint makes the schema impossible to load from outside the network.	The schema is imported by running a command on an application instance already inside the virtual network, using the VM agent rather than SSH.
Azure MySQL blocks trigger and stored procedure creation while binary logging is on.	The server parameter log_bin_trust_function_creators is set to ON during provisioning, before any import is attempted.
An idle Application Gateway bills by the hour and can quietly exhaust a student subscription.	A teardown script, a capped maximum instance count, and a documented practice of destroying the environment after each demonstration.
Managed services increase the cost of moving to another provider.	The database is standard MySQL and the application is plain Python, so only the infrastructure layer is Azure-specific and would need rewriting.

15. Conclusion and Future Scope

A working three-tier database application has been designed for migration to Microsoft Azure. The design uses infrastructure as a service for compute and platform as a service for data, spreads across two Availability Zones for fault tolerance, scales elastically with demand, holds no credential in source control, and is defined entirely as infrastructure code that compiles and validates.

The application itself is unchanged. That is the clearest evidence that separating the data access layer from the interface in the original database project was the right decision: because every SQL statement lives in one module and every credential in one configuration file, moving the database to a managed cloud service became a configuration change rather than a rewrite.

The work also addressed a weakness the original project carried, namely a password stored in plain text inside the repository. The cloud design removes it entirely in favour of a vault and a managed identity.

15.1 Future scope

- Containerise the application and run it on Azure Kubernetes Service, which would improve density and make rolling deployments simpler.
- Add read replicas so that reporting queries do not compete with booking transactions.
- Introduce a continuous deployment pipeline in GitHub Actions, so that a commit to the main branch deploys itself after passing tests.
- Extend the deployment to a second Azure region for disaster recovery beyond a single geography.
- Replace the password-based database login with Microsoft Entra authentication, removing the last stored secret.

16. Repository and Appendix

16.1 Repository

The complete project, including the application source, the database scripts, the infrastructure templates and this documentation, is available at:

```
https://github.com/<your-username>/hotel-management-azure
```

Replace the placeholder above with the repository URL before submission. The same URL must be set as the repositoryUrl parameter in main.parameters.json, because cloud-init clones the repository when each virtual machine boots.

16.2 Deployment commands

From the infra directory, with the Azure CLI installed and authenticated:

```
az login
cp main.parameters.json my.parameters.json      # then fill in the placeholders
PARAM_FILE=my.parameters.json ./deploy.sh
```

To remove everything afterwards:

```
./destroy.sh rg-hotel-prod
```

16.3 Verifying the templates without deploying

The templates can be compiled without an Azure subscription, which is how they were checked for this report:

```
az bicep build --file main.bicep --outfile /tmp/main.json
shellcheck deploy.sh load-schema.sh destroy.sh
```

16.4 Default accounts

The application ships with two accounts for demonstration. Both passwords must be changed before the system is used with real guest data.

Username	Password	Role
`admin`	admin123	Admin
`reception`	admin123	Receptionist